

Azure Auto-Documentation and Diagramming Framework

Architecture Reference: v3 (Public Edition)

System	doc-generator, Azure infrastructure documentation and diagramming pipeline
Version	2.0.0
Document revision	v3-public, August 2026
Status	v3 - Public Edition
Audience	Cloud architects, platform engineers, technical reviewers

Context and Role

The framework was designed and built by the author as part of consulting delivery for regulated-industry Azure estates. This public reference describes that work.

TODO-OTAZKA: potvrdit obdobi, rozsah role a presnou formulaci.

A note on names and numbers. This document describes a production system built for real regulated-industry Azure estates. All subscription, company, and project names are placeholders (estate-*, contoso-demo, region-a/region-b). All JSON samples are synthesized with placeholder data. Every quantitative claim (test counts, benchmark timings, inventory sizes) was re-measured against the repository on 2026-08-13; none is estimated.

1. Executive Summary

1.1 Problem

Azure infrastructure documentation (inventory workbooks, architecture diagrams, and compliance reports) is usually produced by hand. Each subscription costs a cloud architect hours of Azure Portal navigation, copy-paste into spreadsheets, and manual Visio or draw.io drawing. The results are inconsistent across teams, stale within weeks, and unreproducible: when the infrastructure changes, the whole effort starts over. For audits in regulated environments this is a real liability, because nobody can prove the documentation matches the deployed estate.

1.2 Solution

doc-generator is a three-phase pipeline that touches Azure exactly once at collection time and runs fully offline afterwards. It converts a live Azure subscription into structured documentation artifacts:

- Snapshot.** A single Azure CLI collection run produces `snapshot.json`, capturing the full inventory: network, compute, storage, security, RBAC, policies, and diagnostic settings.
- Excel.** The snapshot is transformed into a 30-sheet workbook with conditional formatting, cross-references, and declarative compliance evaluation.
- Diagrams.** The same snapshot feeds a four-stage rendering pipeline that produces hub-and-spoke architecture diagrams as SVG and PNG, with deterministic layout and orthogonal edge routing.

All downstream phases operate entirely offline from the snapshot artifact. No Azure API call is made after collection. For a fixed toolchain and font set, the same snapshot produces byte-identical

outputs. SVG geometry is identical unconditionally; rasterization depends on the cairosvg and font versions.

1.3 Primary Quality Drivers

Quality attribute	Manifestation
Correctness	Topology classification, routing, and geometry verified by 382 automated tests across 7 real Azure subscription snapshots
Determinism	No randomness anywhere; fixed toolchain and font set produce byte-identical output, while SVG geometry remains identical across toolchains; safe for diffing and audit trails
Inspectability	Each pipeline stage writes an intermediate JSON artifact; any stage can be inspected and re-run independently
Recoverability	A <code>--step N</code> flag restarts from any intermediate stage; <code>input_hash</code> lineage detects upstream changes
Minimal dependencies	Extraction, routing, and layout use the Python standard library only; external dependencies are isolated to the final rendering step
Operational simplicity	Single process, sequential execution; no daemon, no server, no database

1.4 Outcome

The pipeline replaced a multi-day manual process with a single CLI invocation per subscription. Diagrams are consistent across teams and reproducible for audits. When infrastructure changes, re-running the pipeline regenerates all artifacts in about a second for the most complex documented estate (multi-region, two hubs; median 1.3 s wall clock over five consecutive runs, worst observed 2.05 s).

2. Purpose and Scope

2.1 The Three Phases

Phase	Input	Output	Entry point
1 — Snapshot	Azure subscription (live)	<code>snapshot.json</code> + CSV projections	<code>scripts/snapshot.sh</code>
2 — Excel	<code>snapshot.json</code>	<code>.xlsx</code> workbook (30 sheets)	<code>scripts/build_docs.py</code>
3 — Diagrams	<code>snapshot.json</code>	<code>.svg</code> + <code>.png</code> (pipeline) or <code>.drawio</code> (legacy)	<code>run_pipeline.py</code> or <code>scripts/build_diagrams.py</code>

Phase 1 also emits per-domain CSV projections for spreadsheet-friendly review.

Phases 2 and 3 never contact the Azure API. The sole data source is the `snapshot.json` artifact produced by Phase 1. This is the core architectural commitment of the system: **collect once, reuse many times**.

2.2 Two Diagram Generators

Two generators coexist by design:

- **Legacy:** `scripts/build_diagrams.py` — a monolithic four-class generator that emits `.drawio` XML. Standard library only. Retained for comparison and debugging.
- **Next-gen:** `run_pipeline.py` — a multi-pass, config-driven pipeline that emits SVG and PNG through four separate processors with intermediate JSON artifacts.

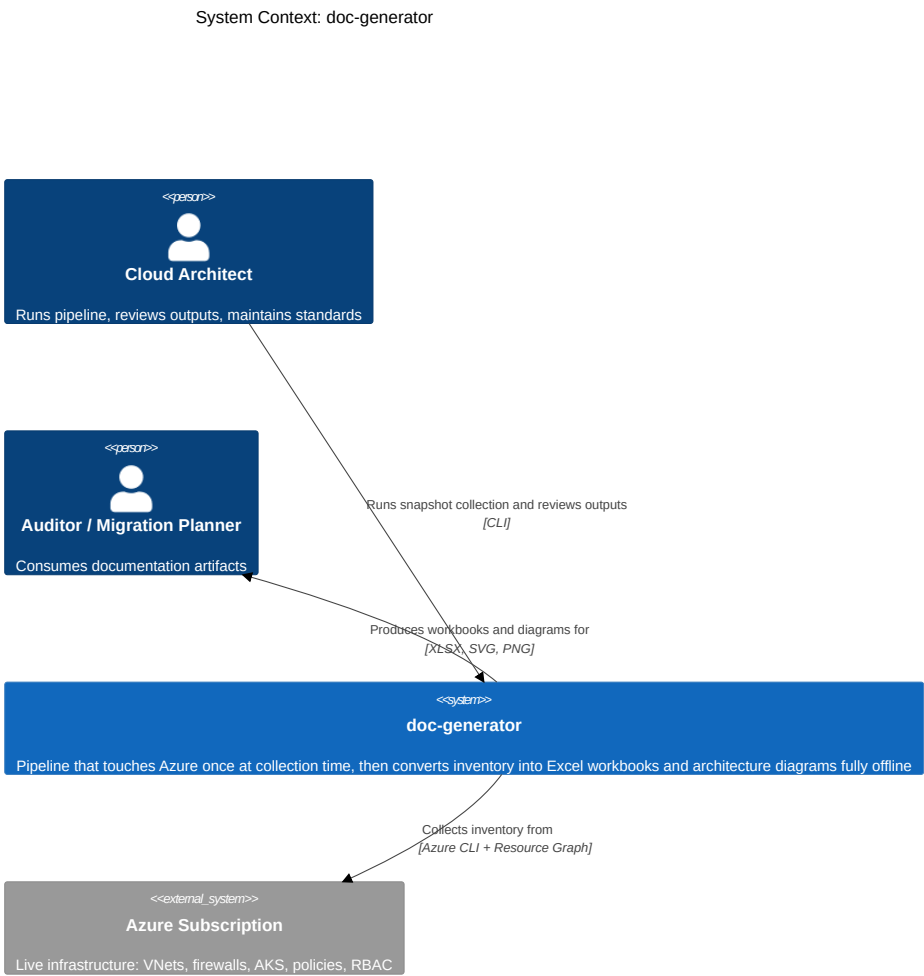
This document covers both but emphasizes the next-gen pipeline, which is the primary architecture going forward.

2.3 Out of Scope

The framework documents infrastructure; it does not deploy it. Deployment is handled by a companion system (a declarative Azure deployment framework), and a future convergence of the two is sketched in Section 14.

3. System Architecture (C4)

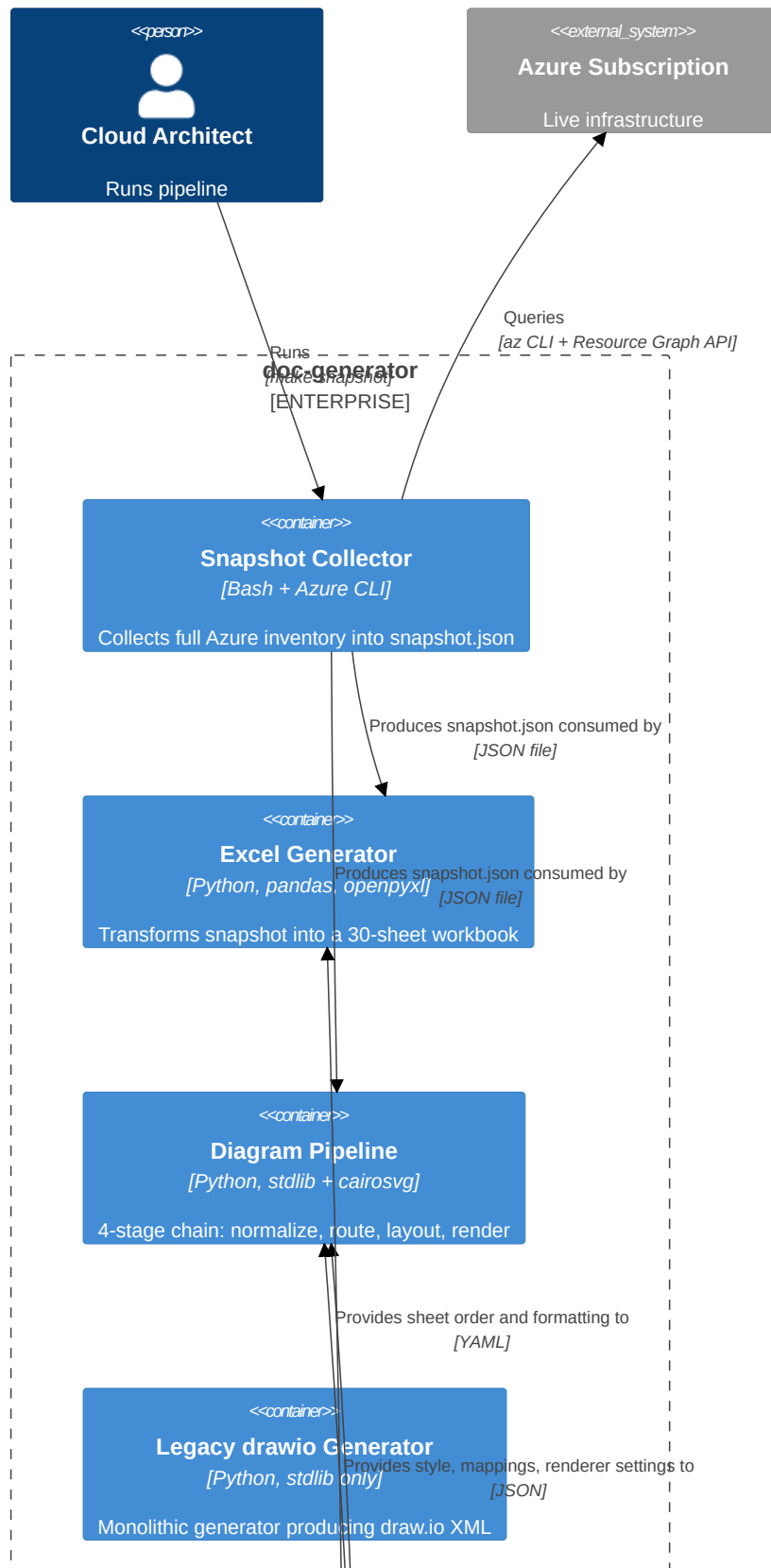
3.1 Level 1: System Context

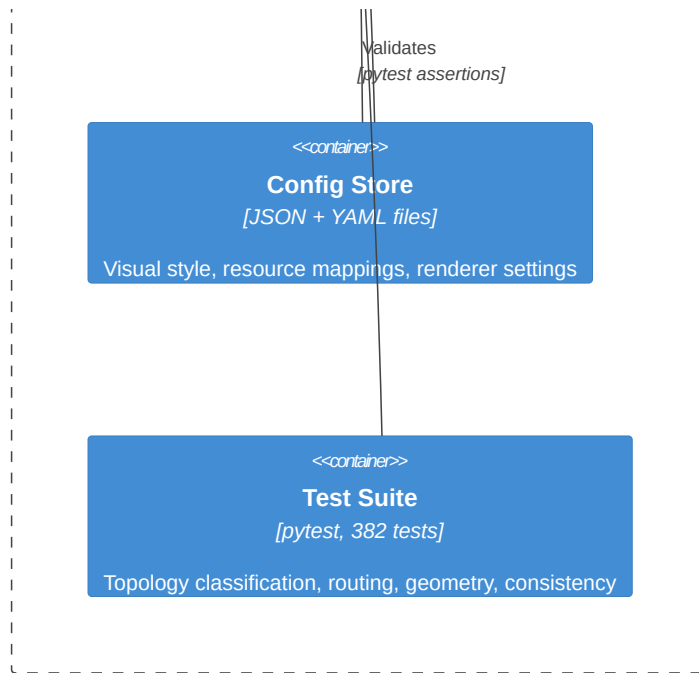


Element	Type	Description
Cloud Architect	Person	Runs the pipeline, reviews outputs, maintains architecture standards
Auditor / Migration Planner	Person	Consumes artifacts for audits, migration planning, compliance
Azure Subscription	External system	Live infrastructure, queried once via Azure CLI and Resource Graph

3.2 Level 2: Containers

Container Diagram: doc-generator

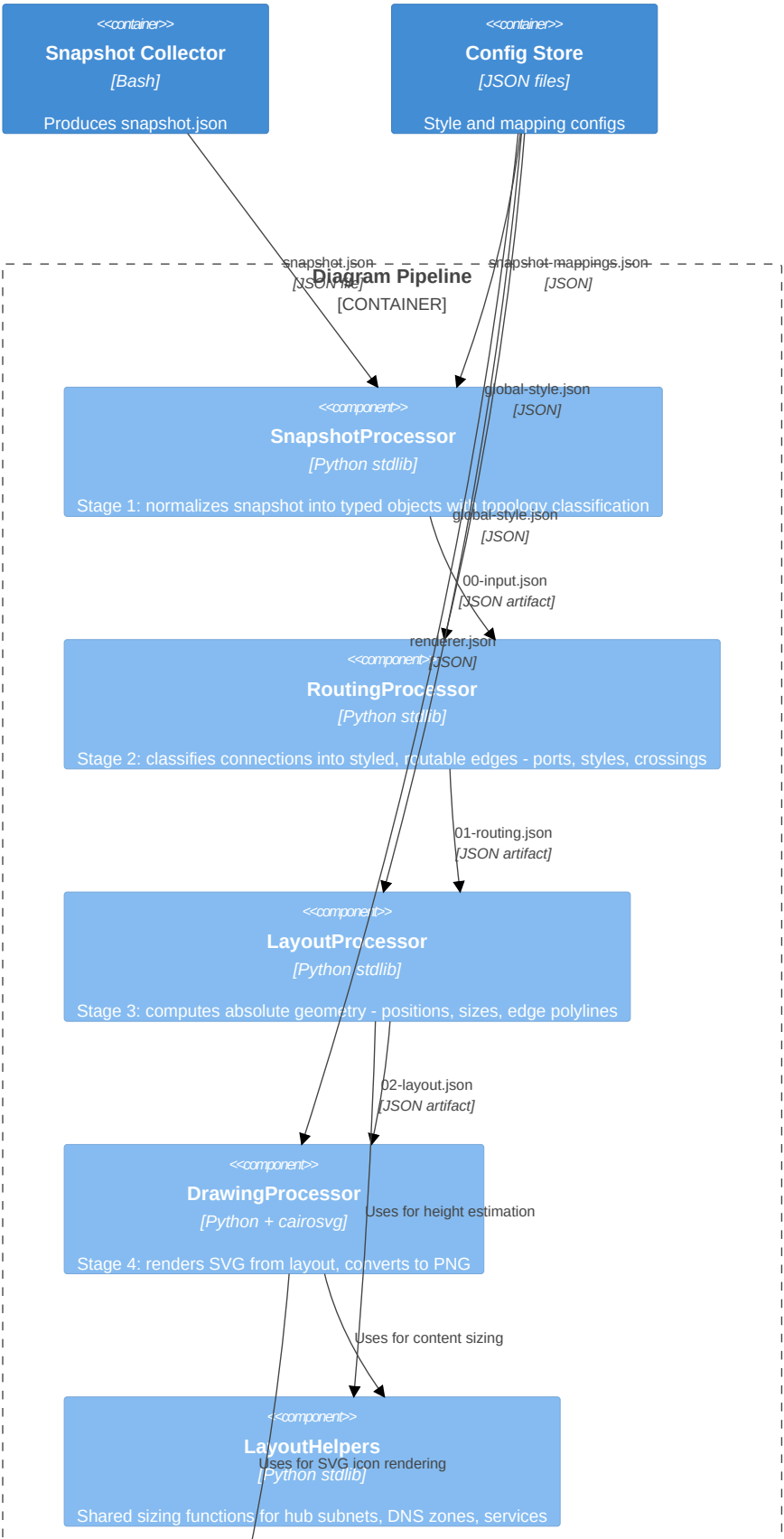


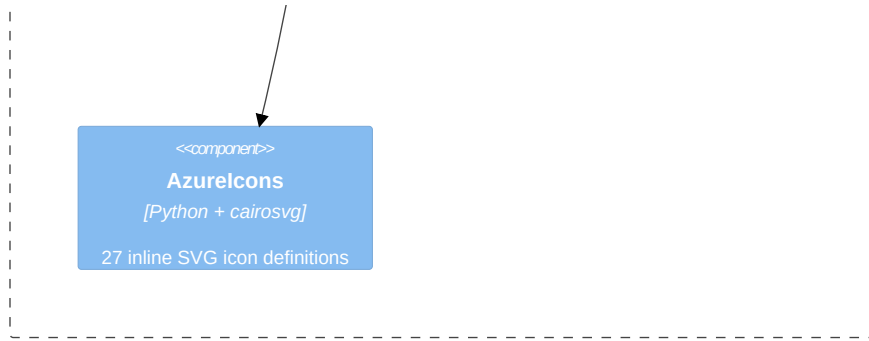


Container	Technology	Responsibility
Snapshot Collector	Bash + Azure CLI	Collects full Azure inventory into <code>snapshot.json</code>
Excel Generator	Python, pandas, openpyxl	Transforms snapshot into a 30-sheet workbook with compliance evaluation
Diagram Pipeline	Python, stdlib + cairosvg	Four stages: normalize, route, layout, render (SVG + PNG)
Legacy drawio Generator	Python, stdlib only	Monolithic generator producing draw.io XML
Config Store	JSON + YAML	Visual design system, extraction rules, renderer settings
Test Suite	pytest	382 tests across 7 real subscription snapshots

3.3 Level 3: Diagram Pipeline Components

Component Diagram: Diagram Pipeline

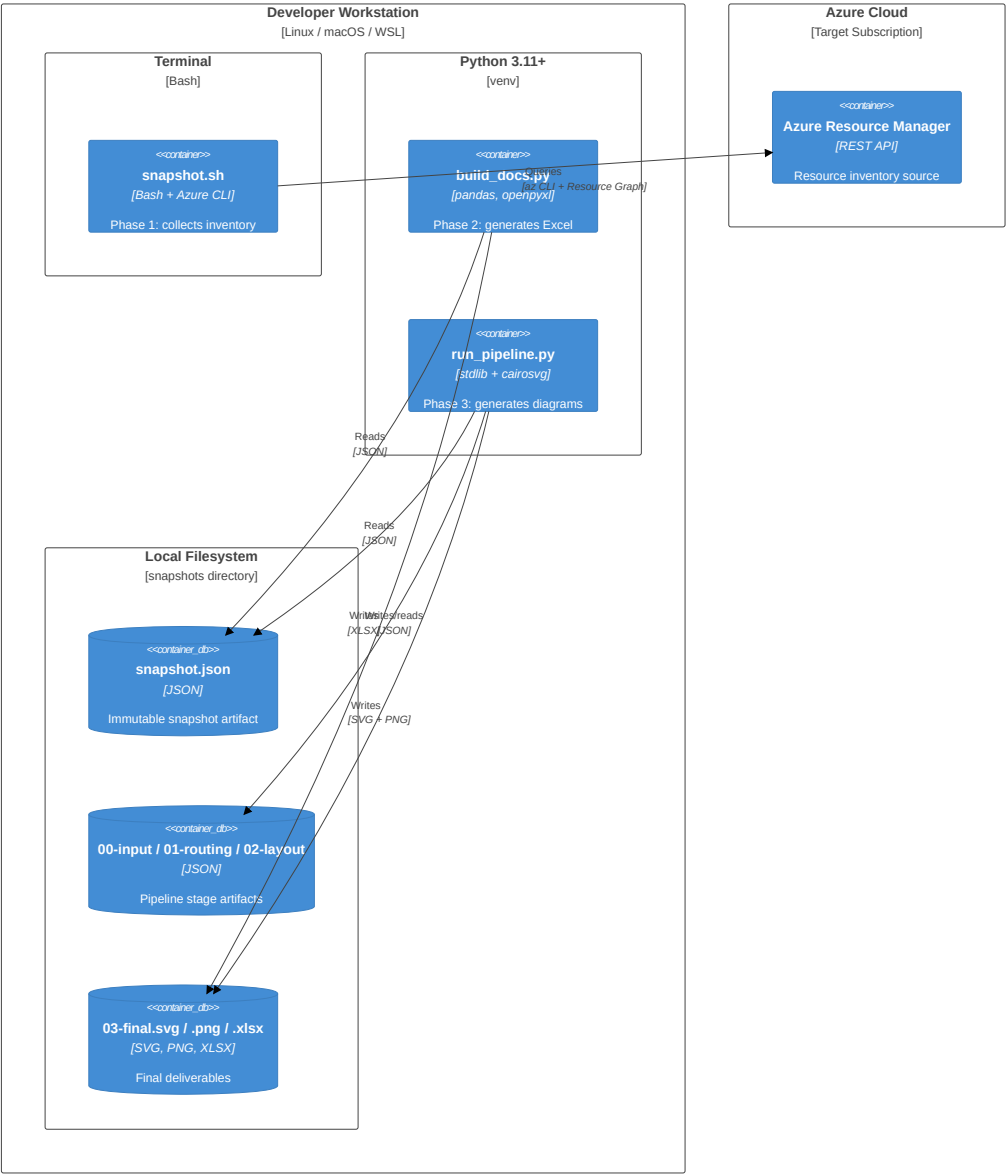




Component	Technology	Responsibility
SnapshotProcessor	Python stdlib	Stage 1: normalizes raw <code>snapshot.json</code> into typed objects with hub/spoke/transit/isolated classification
RoutingProcessor	Python stdlib	Stage 2: classifies connections into styled, routable edges - ports, styles, crossings
LayoutProcessor	Python stdlib	Stage 3: computes absolute geometry — positions, sizes, orthogonal edge polylines
DrawingProcessor	Python + cairosvg	Stage 4: renders the SVG canvas, rasterizes to PNG at target DPI
LayoutHelpers	Python stdlib	Shared sizing functions both layout and drawing must agree on
Azurelcons	Python + cairosvg	27 inline SVG icon definitions with a deterministic shape fallback

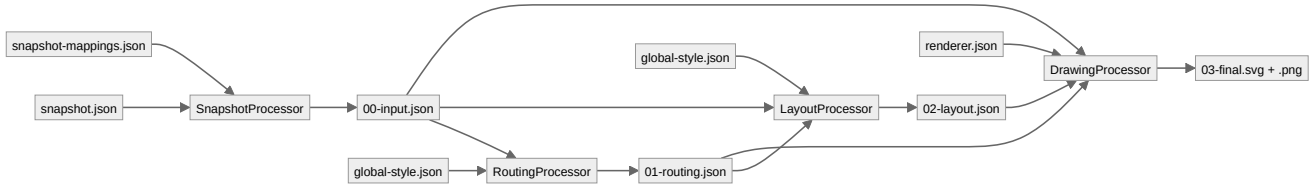
3.4 Level 4: Deployment View

Deployment Diagram: doc-generator

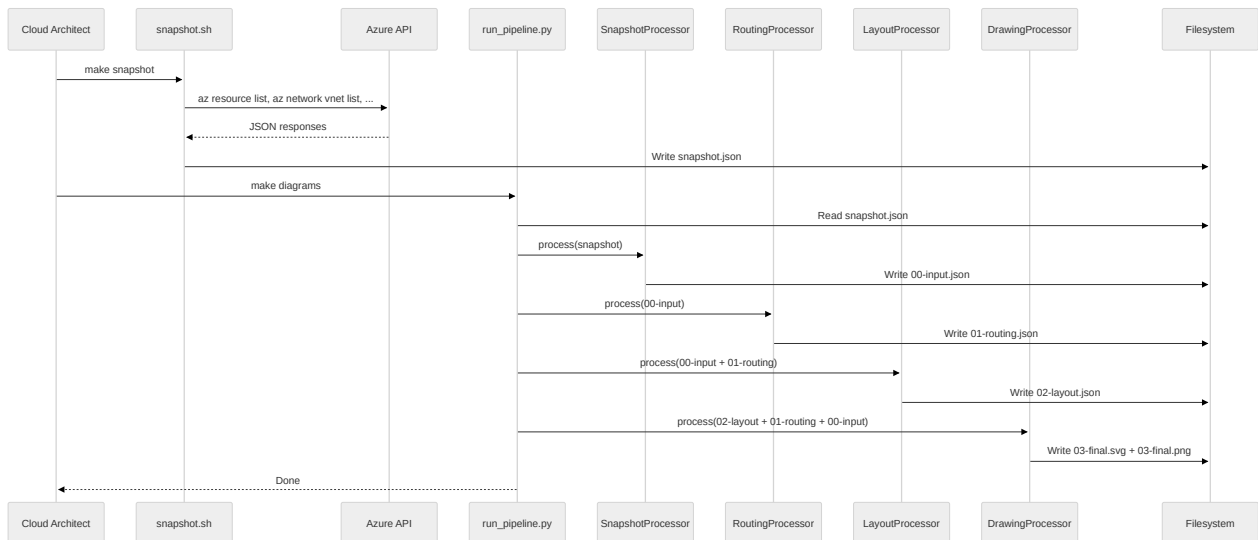


There is deliberately no server, no CI/CD dependency, and no database. The sole execution environment is a workstation with Python 3.11+ and the Azure CLI. Azure is contacted read-only, exactly once, during Phase 1.

3.5 Data Flow: Next-Gen Pipeline

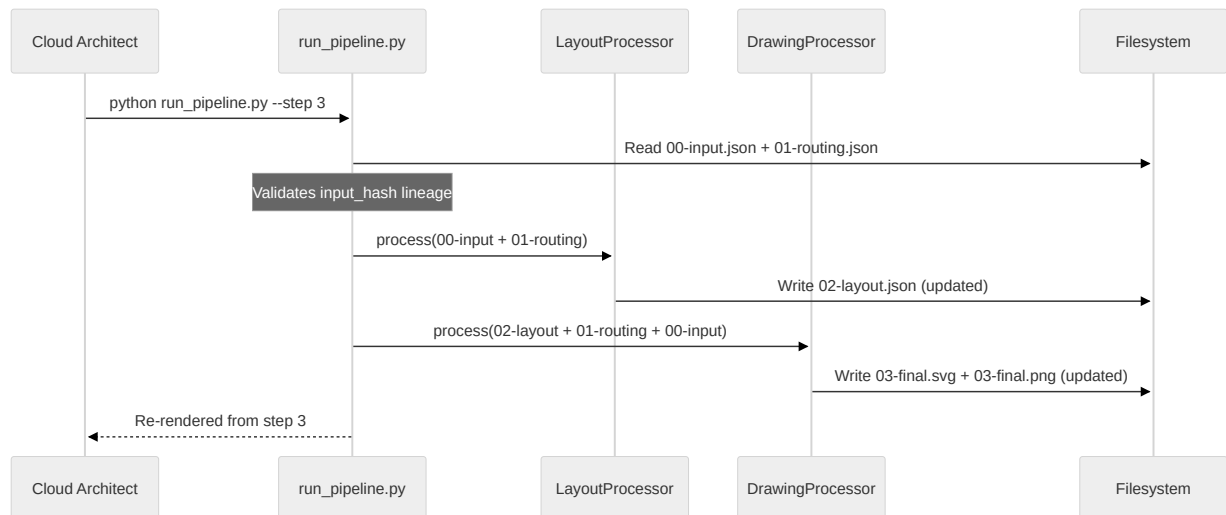


3.6 Key Scenario: Full Pipeline Run



3.7 Key Scenario: Mid-Pipeline Re-Run

A layout fix does not require re-running extraction or routing:



4. The Four-Stage Diagram Pipeline

Each stage is a separate Python class with a single `process()` entry point. Each stage reads JSON from disk, transforms it, and writes JSON (or SVG/PNG) to disk. The orchestrator (`run_pipeline.py`) contains no domain logic; it wires stages together and implements the `--step N` resume capability.

A hard dependency rule applies: **stages 1 to 3 depend only on the Python standard library** (`json`, `pathlib`, `re`, `logging`, `hashlib`, `math`). Only stage 4 crosses the external dependency boundary, and only for SVG-to-PNG conversion via `cairosvg`. This keeps extraction and layout logic portable and trivially testable.

4.1 Stage 1: SnapshotProcessor (normalize)

Converts the raw, deeply nested Azure snapshot into a flat object/connection model that downstream stages consume without any Azure-specific knowledge. This is where hub detection and topology classification happen (algorithm in Section 6.1).

Processing runs in twelve ordered steps:

Step	Responsibility
1	Parse VNets with subnets and peerings
2	Classify VNets (two-pass: roles first, then hub attachment)
3	Extract hub resources (Application Gateway, Firewall, VPN Gateway, NAT Gateway)
4	Extract spoke resources (AKS clusters, internal load balancers)
5	Create peering connection objects
6	Discover external resources (Front Door, Traffic Manager, ExpressRoute, DNS)
7	Parse gateway connections (VNet-to-VNet, site-to-site VPN, ExpressRoute)
8	Build private-endpoint edges from graph insights
9	Extract private DNS zones with categorization
10	Map DNS zones to VNets
11	Extract supporting resources (storage accounts, key vaults, public IPs)
12	Serialize everything to <code>00-input.json</code>

Selected invariants, enforced by tests:

- Object IDs are stable across runs for the same snapshot.
- Self-loop private endpoints (endpoint and target in the same VNet) are filtered out.
- Hub detection does not require a firewall subnet; a gateway subnet alone is sufficient. This is what makes the firewall-less Distributed Native profile work (Section 8).

4.2 Stage 2: RoutingProcessor (classify edges)

Assigns each connection its attachment ports (N/S/E/W), a visual style, and container-crossing metadata. Every connection in `00-input.json` produces exactly one edge in `01-routing.json`; edge IDs are stable (`edge-001`, `edge-002`, ...).

Port assignment follows hub-top layout semantics:

Edge type	From port	To port	Rationale
Hub to spoke	S	N	Hub exports downward
Cross-region (primary to DR)	S	N	Vertical inter-region corridor
Hub to transit	E	W	Horizontal transit lane
External (VPN/ER) to hub	S	N	Enters hub from the top
Spoke to spoke	E	W	Lateral, routed below the grid

Each edge type carries a fixed style (color, width, dash pattern, arrowheads) from a single palette, so every diagram in an organization reads the same way. Disaster recovery peerings are highlighted in red regardless of type.

4.3 Stage 3: LayoutProcessor (geometry)

The geometry stage is about 1,800 lines. It computes absolute x/y/width/height for every element and resolves every edge into a final orthogonal polyline. After this stage, no spatial decision remains: the renderer only draws.

The canvas is organized as six vertical zones:

Zone	Name	Height	Content
Z0	Title	56 px	Subscription name, region info
Z1	External connectivity	80-120 px	Internet, Front Door, ER, DNS
Z2a	Primary region	dynamic	Hub + spoke grid
Z2b	DR region (optional)	dynamic	Region separator + DR hub + spokes
Z3	Supporting services	computed	Storage, public IPs, DNS, governance
Z4	Legend	48 px	Connection-type legend
Z5	Metadata footer	20 px	Timestamp, resource counts

Sizing is content-aware, not percentage-based. The hub container stretches to span the spoke grid beneath it (`hub_w = max(spoke_grid_w, HUB_W_MIN)`); the spoke grid uses 2, 3, or 4 columns depending on spoke count; page width is derived from actual content extents. The result is that a DR region sits visually close to the primary region instead of floating in empty space.

The layout also publishes explicit **routing channels** — left/right lane corridors, row-gap bands, and a dedicated inter-region corridor — that edge polylines are routed through. Channels are exported in `02-layout.json`, which makes routing decisions externally verifiable (Section 9.3).

4.4 Stage 4: DrawingProcessor (render)

A pure renderer. It builds an in-memory SVG canvas from the layout, then rasterizes it once via `cairosvg.svg2png()` at the configured DPI (default 150). Both artifacts are written side by side: `03-final.svg` (lossless, zoomable, diffable) and `03-final.png` (for embedding into the Excel workbook and slide decks).

Two rules keep this stage honest:

1. **It never calculates positions.** All coordinates come from `02-layout.json`. If something does not fit, the bug is in the layout stage by definition.
2. **DPI scaling is applied exactly once**, at the final SVG-to-PNG conversion. All drawing happens in logical 96-DPI coordinates.

Icons are emitted as inline SVG groups from a library of 27 Azure resource icons stored as path data. When `cairosvg` or the system Cairo library is missing, icons degrade deterministically to colored shapes with letter labels, and the SVG is still produced.

4.5 Shared Sizing Helpers

Layout and drawing must agree on content heights: if layout reserves 400 px for a hub and the renderer draws 500 px of content, the diagram breaks. All height estimation (application-gateway subnet blocks, firewall blocks, DNS zone bars, services zone, storage grids) lives in one shared module that both stages import. These functions are the single source of truth for sizing.

5. Data Contracts

Every stage boundary is a documented JSON contract. The contracts are the reason the pipeline is debuggable: each intermediate file is human-readable and can be diffed, inspected, or hand-authored. All samples below are synthesized with placeholder data.

5.1 `snapshot.json`: Phase 1 Output

Top-level sections: `snapshot` (metadata), `resources` (full ARM export), `inventory` (normalized by domain: network, compute, storage, security), `diagnostics`, `roleAssignments`, `roleDefinitions`, `policies`, `privateConnectivity`, `rgSummary`, `graphInsights`, `warnings`. A standalone schema reference (about 1,900 lines) documents every field, so external consumers — human or automated — can use the snapshot without project-internal knowledge.

Collection problems surface as structured entries in `warnings.json`, never as silent omissions. This distinction matters for audit use: an empty section means "verified empty", not "failed to collect".

5.2 `00-input.json` : Normalized Infrastructure Model

Produced by stage 1. Describes WHAT exists and WHAT connects. No positions, no sizes.

```

{
  "metadata": {
    "processor": "snapshot",
    "version": "1.0",
    "generated_at": "2026-08-13T10:00:00Z",
    "input_hash": "3f2a9c81d4e07b65"
  },
  "diagram": {
    "name": "contoso-demo",
    "title": "Contoso Demo - Azure Infrastructure"
  },
  "topology": {
    "hub_count": 1,
    "spoke_count": 4,
    "is_multi_region": false,
    "primary_region": "region-a",
    "vnet_classification": {
      "vnet-hub-prod": {
        "role": "hub",
        "region_role": "primary",
        "hub_name": "",
        "location": "region-a"
      },
      "vnet-spoke-app-prod": {
        "role": "spoke",
        "region_role": "primary",
        "env_class": "production",
        "hub_name": "vnet-hub-prod",
        "location": "region-a"
      }
    }
  },
  "objects": [
    {
      "id": "vnet-hub-prod",
      "type": "vnet",
      "label": "vnet-hub-prod",
      "children": ["subnet-firewall", "subnet-gateway", "subnet-appgw"],
      "properties": {
        "address_space": "10.0.0.0/16",
        "resource_group": "rg-hub-prod",
        "role": "hub"
      }
    }
  ],
  "connections": [
    {
      "from": "vnet-hub-prod",
      "to": "vnet-spoke-app-prod",
      "type": "peering",
      "label": "VNet Peering",
      "bidirectional": true,
      "properties": { "state": "Connected" }
    }
  ],
  "summary": {
    "total_resources": 100,
    "total_rgs": 8,
    "subscription_name": "contoso-demo"
  }
}

```

```
}  
}
```

Twenty object types are recognized (vnet, subnet, firewall, app_gateway, vpn_gateway, nat_gateway, aks, load_balancer, front_door, traffic_manager, expressroute, dns_zone, private_dns, storage_account, key_vault, route_table, nsg, public_ip, private_endpoint, comms_service) and ten connection types (peering, cross_region_peering, spoke_to_spoke_peering, s2s_vpn, vnet2vnet, expressroute, private_endpoint, dns_link, traffic, internal).

5.3 01-routing.json: Edge Classification

Produced by stage 2. One edge per connection, with ports, style, and diagnostics:

```
{  
  "edges": [  
    {  
      "id": "edge-001",  
      "from": "vnet-hub-prod",  
      "to": "vnet-spoke-app-prod",  
      "from_port": "S",  
      "to_port": "N",  
      "type": "peering",  
      "label": "VNet Peering",  
      "style": { "color": "#0050ef", "width": 2, "dashed": true, "arrow": false },  
      "crosses_containers": []  
    }  
  ],  
  "node_edge_counts": {  
    "vnet-hub-prod": { "N": 1, "S": 5, "E": 0, "W": 0, "total": 6 }  
  }  
}
```

5.4 02-layout.json: Spatial Layout

Produced by stage 3. Absolute geometry for every object, plus fully resolved edge polylines and the routing channels they were routed through:

```
{
  "canvas": { "width": 1500, "height": 2922, "background": "#FFFFFF" },
  "routing_channels": {
    "primary": {
      "hub_name": "vnet-hub-prod",
      "left_lane_x": 20,
      "right_lane_x": 1100,
      "first_row_lane_y": 560,
      "grid_mid_x": 560
    },
    "inter_region": { "lane_x": 8 }
  },
  "edge_routes": [
    {
      "id": "edge-001",
      "from_id": "hub:vnet-hub-prod",
      "to_id": "spoke:vnet-spoke-app-prod",
      "points": [[240.0, 544.0], [240.0, 600.0]],
      "label_bias": 0.4,
      "label_text": "VNet Peering"
    }
  ],
  "objects": [
    {
      "id": "hub:vnet-hub-prod",
      "type": "hub",
      "x": 40, "y": 144, "width": 680, "height": 400,
      "label": { "text": "vnet-hub-prod", "x": 50, "y": 152 },
      "sublabel": { "text": "10.0.0.0/16", "x": 50, "y": 168 }
    }
  ]
}
```

5.5 Lineage

Every artifact records a 16-hex-character SHA-256 prefix of its input:

```
snapshot.json
-> 00-input.json      (input_hash = hash of snapshot.json)
-> 01-routing.json    (input_hash = hash of 00-input.json)
-> 02-layout.json     (input_hash = hash of 00-input.json + 01-routing.json)
-> 03-final.svg/png   (from 02-layout + 01-routing + 00-input)
```

When a stage is re-run with `--step N`, the orchestrator compares hashes and warns if upstream artifacts changed since the cached intermediates were produced. Stale-input bugs — the classic failure of multi-stage build systems — are detected, not silently propagated.

6. Key Design Patterns

6.1 Two-Pass Hub Detection and VNet Classification

Correct diagram generation stands or falls with hub detection. Both generators implement the same algorithm.

Pass 1 — role assignment. For each VNet, inspect its subnet names (lowercased):

```

has_fw_subnet      = "azurefirewallsubnet" present
has_gw_subnet      = "gatewaysubnet" present
has_appgw_subnet   = any subnet name containing "appg"

if has_fw_subnet:
    role = hub
elif has_gw_subnet:
    role = hub    (supports the Distributed Native
profile)
elif no peerings:
    role = isolated
else:
    role = spoke

```

Hub candidate scoring. When one region contains several hub-capable VNets, a deterministic score selects the primary hub; the rest are demoted to transit (if peered) or isolated:

```

score = 8 * has_fw_subnet + 4 * has_appgw_subnet + 2 * has_gw_subnet
score += min(peering_count, 3) + min(subnet_count, 4)

```

Pass 2 — hub attachment. Only after all roles are assigned does each spoke get its `hub_name` by following its peerings. Running the passes in one sweep caused a real ordering bug in an early version: a spoke could try to attach to a VNet that had not yet been classified as a hub. The two-pass split fixed it permanently.

Environment class is derived from naming conventions via regular expressions (production, nonprod, DR), which in turn drives the diagram color coding.

6.2 Orthogonal Edge Routing Through Lane Corridors

Edges are horizontal/vertical polylines only, routed through explicit corridors so they never cross container interiors:

- **Hub to first-row spoke:** a straight vertical drop, one visual trunk per spoke column. Edges sharing a column deliberately produce identical coordinates and render as a single line.
- **Hub to lower rows:** vertical exit, horizontal to the left or right outer lane, vertical down the lane, horizontal into the spoke column.
- **Spoke to spoke:** routed below the grid.
- **Cross-region and VNet-to-VNet:** a dedicated corridor on the far left of the canvas; site-to-site VPN and ExpressRoute edges that share this corridor are offset by 12 px so they remain individually visible.
- **Private endpoint to storage account:** two modes — a direct four-point route when the source sits just above the services zone, and a six-point detour around the right edge of the canvas when hub or spoke content lies between. The detour is what guarantees the polyline never enters a container body.

Edge labels are placed at a configurable fraction along the polyline. A clearance sweep tests candidate positions against every container's bounding box (including the edge's own endpoints) and picks the first position whose label overlaps nothing.

6.3 Config-Driven Everything

Three permanent config files externalize every visual and extraction decision:

File	Controls
<code>global-style.json</code>	Spacing system, typography, container styles, connection styles, canvas minimums, color palette
<code>renderer.json</code>	Antialiasing, shadows, legend, DPI, output format, debug overlays
<code>snapshot-mappings.json</code>	42 resource-type mappings onto the 27 icon definitions; several resource types share a glyph; connection-inference rules; per-type property extraction; 9-entry ignore list

The rule is absolute: **processor code is never edited for visual or extraction adjustments**. Visual tuning is a config edit plus a re-run from the relevant stage.

6.4 Determinism and Idempotency

There is no randomness anywhere in the pipeline: no force-directed layout, no spring models, no random seeds. Icon fallbacks are seeded by icon key. Object and edge IDs are stable. Outputs are byte-identical for a fixed toolchain and font set; the SVG geometry is identical unconditionally (rasterization depends on the cairosvg and font versions). This means:

- diagram diffs between two snapshots show real infrastructure changes, nothing else;
- audit reproducibility is trivial — re-run and compare;
- caching via `input_hash` is safe.

6.5 Fallback-First External Dependencies

The only external rendering dependency (cairosvg plus the native Cairo library) is wrapped in graceful degradation: without it, the pipeline still produces the complete SVG, and icons fall back to deterministic colored shapes. The legacy generator needs no third-party packages at all. A documentation tool that fails to install on a locked-down workstation would not be used; this one runs anywhere Python runs.

7. Phase 2 — Excel Workbook Generation

The workbook generator transforms the same `snapshot.json` into a 30-sheet Excel dossier using pandas and openpyxl. Sheet order and formatting are config-driven. Sheets cover the full estate: overview and metadata, resource-group inventory, network (VNETs, DNS, entry points, firewall rules and policies, load balancers, NSG matrices, traffic paths, routing and peering), AKS clusters and node pools, certificates, storage (accounts, containers, file shares, queues, tables), private endpoints, public IPs, policy assignments, diagnostics coverage, RBAC assignments and role definitions, compliance results, collection warnings, and a discovery report.

Two sheets deserve mention as governance features:

- **Compliance** — evaluates declarative rules from `policy_rules.yaml` against the snapshot, turning the workbook into a lightweight audit report.
- **Discovery Report / Collection Warnings** — surface what was NOT collected and why, so reviewers can distinguish absence of data from absence of resources.

The architecture diagram PNG produced by Phase 3 is embedded into the Overview sheet at its native pixel dimensions, closing the loop: one snapshot, one workbook, one diagram, all consistent.

8. Architecture Profile Integration

The pipeline is standards-aware. Estates in scope follow one of two governed network profiles, and the generator handles both.

The Centralized Inspection profile

- Hub-and-spoke with a central Azure Firewall in the hub.
- All spoke traffic hairpins through the firewall (default route 0.0.0.0/0 to FW).
- Spoke-to-spoke peering is prohibited.
- DNS via firewall DNS proxy.

Diagram impact: the hub renders firewall, gateway, and application-gateway subnets; no spoke-to-spoke edges appear; a routing summary bar shows the enforced default route.

The Distributed Native profile

- Hub-and-spoke without a central firewall; the hub carries the gateway subnet and a DNS Private Resolver.
- East-west traffic flows over direct VNet peering, governed by NSG rules.
- Spoke-to-spoke peering is allowed and expected.

Diagram impact: no firewall subnet, spoke-to-spoke peering edges appear, no routing summary bar.

The classification algorithm supports both by design: hub status requires only a gateway subnet, not a firewall (Section 6.1). One of the seven test snapshots (`estate-06`) exists specifically to pin the firewall-less profile in the test suite.

9. Testing and Verification

9.1 Test Suite

All numbers below were measured by running the suite on 2026-08-13 (result: 271 passed, 111 skipped, 13.5 seconds, exit code 0). Skips are by design: snapshot- specific assertions are parametrized across all snapshots and skip where they do not apply.

Test module	Tests	Focus
Pipeline suite	326	Schema validation, unique object IDs, no self-loops, role classification, edge counts, valid ports and types, canvas geometry, cross-step consistency, per-snapshot topology assertions
PNG geometry suite	54	Rendered-output geometry: rendered boxes and edges match the layout contract
Icon suite	2	Icon library integrity and rendering
Total	382	

9.2 Test Registry: Seven Real Estates

The suite is parametrized over seven snapshots of real Azure subscriptions (anonymized here; topology figures verified by running the pipeline over each snapshot on 2026-08-13):

Scale bands are small (under 100 resources), medium (100-250), and large (250-500).

Snapshot	Scale	Topology	Distinctive coverage
estate-01	large	multi-region with two hubs	ExpressRoute, S2S VPN, private endpoints, cross-region peering
estate-02	large	single region with one hub	S2S VPN
estate-03	large	single region with one hub plus a transit VNet	VNet-to-VNet gateway links
estate-04	medium	single region with one hub	spoke-to-spoke peering
estate-05	medium	single region with one hub	DNS-zone-heavy estate
estate-06	medium	single region with a firewall-less hub (Distributed Native profile)	spoke-to-spoke peering, private endpoints
estate-07	small	minimal hub-and-spoke	smallest topology in the suite

Exact per-estate inventories (resource, VNet, and edge counts) are deliberately withheld from the public edition.

Testing against real estates rather than toy fixtures is deliberate. Every quirk in the suite — a transit VNet demoted from hub status, a firewall-less hub, a private endpoint crossing spokes — was first observed in production infrastructure.

9.3 The Routing Oracle

Visual inspection does not scale as routing complexity grows. A dedicated verifier runs six geometric checks against `02-layout.json`:

Check	Detects
1 — Hub exit stubs	Edges in the same spoke column exiting the hub at different x positions
2 — Zigzag jog	First-row edges that are not straight vertical drops
3 — Shared segments	Distinct edges overlapping on more than three identical points
4 — Edge outside hub	Edge origins outside the hub's horizontal bounds
5 — Edge crosses object	Any polyline segment intersecting any container body (Liang-Barsky segment/AABB test, plus strictly-inside endpoint probes)
6 — Label overlap (strict)	Any label bounding box intersecting any container, including the edge's own endpoints

Checks 5 and 6 are mandatory gates: a layout that fails them is not correct, no matter how it looks. New routing modes must ship with corresponding verifier cases (ADR-006).

9.4 Cross-Step Consistency

Because stage boundaries are contracts, consistency is testable mechanically: every connection must map to exactly one routed edge; every VNet in the input must have a placed container in the layout; canvas bounds must contain every object. These tests have repeatedly caught contract drift that unit tests within a single stage missed.

10. Performance

One benchmark is used throughout this document: the full four-stage pipeline over the most complex estate in the registry completes in about a second (median), and never exceeded 2.1 seconds in measurement.

Benchmark protocol (2026-08-13): five consecutive runs of `run_pipeline.py` over a sanitized copy of the most complex snapshot in the registry: multi-region, two hubs, the largest diagram the suite produces. Python 3.13, developer laptop (Apple Silicon), 150 DPI output, canvas 1500 x 2922 px.

Run	Pipeline-internal time	Wall clock (incl. interpreter start)
1	1.3 s	1.67 s
2	1.9 s	2.05 s
3	1.1 s	1.22 s
4	1.1 s	1.26 s
5	0.9 s	0.94 s
Median	1.1 s	1.26 s

Output artifacts for this estate: `03-final.png` 695 KB, `03-final.svg` 92 KB. The complete test suite (382 tests, including seven full pipeline executions and PNG geometry verification) runs in 13.5 seconds.

Performance was never the design driver — correctness and determinism were — but the sequential, stdlib-only architecture turned out fast enough that parallelism was never worth its complexity (ADR-003).

11. Architecture Decision Records

ADR-001: `snapshot.json` as the Sole Source of Truth

Context. Both the workbook and the diagrams need Azure inventory data. Either each phase queries Azure independently, or one collection run feeds everything.

Decision. A single `snapshot.json` is collected once and is the exclusive input to all downstream phases. No Azure API calls after collection.

Consequences. Phases 2 and 3 are fully offline and deterministic. Snapshots can be archived, diffed, and audited. The trade-off: a snapshot is a point-in-time view; documenting changed infrastructure requires re-collection.

ADR-002: Custom Multi-Pass Pipeline over a Generic Diagram Library

Context. Existing options (diagrams-as-code libraries, Structurizr, draw.io plugins) were evaluated. None combined hub-and-spoke topology classification, content-aware sizing, orthogonal lane-corridor routing, and deterministic output from real Azure inventory.

Decision. Build a four-stage pipeline with JSON contracts between stages, each stage independently testable and restartable.

Consequences. Full control over layout and rendering, at higher initial cost. The intermediate artifacts turned out to be the key enabler of debugging, verification, and iterative tuning. New output formats (Mermaid, PlantUML) could consume `02-layout.json` without touching upstream stages.

ADR-003: Sequential Single-Process Execution

Context. The pipeline could parallelize across subscriptions or stages. Typical workload: one to seven subscriptions, typically a few hundred resources each.

Decision. Strictly sequential, single process. Multi-subscription runs are a shell loop.

Consequences. No concurrency bugs, no shared state, no locking. The reference estate completes in about a second (median), so parallelism would optimize a non-problem. If throughput ever matters, parallelism at the subscription level would be simple to add.

ADR-004: Config-Driven Layout and Styling

Context. Early iterations hardcoded spacing, colors, and fonts in processor code. Every visual change meant a code change.

Decision. All visual parameters live in three config files; processors read them at initialization and contain no visual constants.

Consequences. Visual tuning without code changes; configs versioned alongside code. Slight indirection cost when reading processor code.

ADR-005: SVG-First Rendering with PNG Export

Context. The original renderer drew pixels directly with Pillow: approximate text metrics, rasterized icons, output locked to one DPI.

Decision. Build an in-memory vector SVG canvas; rasterize once via cairosvg at the configured DPI; ship both SVG and PNG.

Consequences. Native vector quality for text and icons, zoomable output, single DPI-scaling point. Trade-off: a native-library dependency (Cairo) for PNG export, mitigated by graceful degradation (Section 6.5).

ADR-006: A Geometric Verifier as the Routing Oracle

Context. As routing grew (private-endpoint detours, label clearance, multi-region lane offsets), visual inspection stopped being a reliable correctness check.

Decision. A read-only verifier with six geometric checks is the authoritative post-layout correctness gate. Checks 5 (edge crosses object) and 6 (label overlap) are mandatory and never skipped.

Consequences. New routing modes must include verifier coverage before merging. "Looks right" was replaced by "provably does not intersect".

ADR-007: Keep the legacy generator during the migration

Context. The drawio generator predates the pipeline, and teams still consume drawio output.

Decision. The two generators coexist until the pipeline fully supersedes the legacy path. The legacy generator is retained for comparison and debugging.

Consequences. Migration risk is reduced, and outputs can be cross-checked. The trade-off is dual maintenance of two render paths.

12. Limitations and Failure Modes

12.1 Known Limitations

1. **Grid-adjacency routing.** Hub-to-spoke peering toward non-adjacent spokes shares lane corridors; extremely dense estates could exhaust visually distinct lane positions.
2. **Icon rendering dependency.** Full-fidelity icons need cairosvg plus the native Cairo library; without them, icons degrade to labeled shapes (the diagram remains complete and correct).
3. **Font portability.** Font discovery tries Segoe UI, then Arial, then Helvetica, then a default; typography varies slightly across systems.
4. **Canvas ceiling.** Maximum canvas size is capped (configurable; default 4000 x 3000 px). An estate beyond that would need pagination, which does not exist today.
5. **Point-in-time semantics.** A snapshot documents a moment. There is no drift detection between snapshot and live estate other than re-collecting and diffing.
6. **Single subscription per run.** Cross-subscription topologies (for example, hub in one subscription, spokes in another) must be snapshotted per subscription and joined by the reader.

12.2 Failure Modes by Phase

Failures surface at their stage boundary. Stages fail fast on invalid inputs or missing dependencies, while collection gaps and lineage mismatches appear as structured warnings instead of silent omissions. Intermediate artifacts from completed stages remain available when a later stage fails, and SVG output remains available when PNG rasterization is unavailable.

12.3 Debugging Playbook

Each defect isolates to one stage because the artifacts expose normalization, routing, layout, and rendering in order. Inspection proceeds from `00-input.json` to `01-routing.json`, `02-layout.json`, and `03-final.svg`; processing can then resume from the failed stage while the earlier artifacts remain unchanged.

12.4 Idempotency and Safe Re-Runs

All stages are idempotent; outputs are overwritten in place; there is no accumulated state between runs. The recovery model is two rules:

Stage failed?	Fix the issue, re-run with <code>--step N</code> (earlier outputs are reused from disk)
Snapshot changed?	Re-run from step 1 (<code>input_hash</code> differs; everything downstream re-processes)

13. Lessons Learned

1. **Intermediate artifacts pay for themselves immediately.** This design choice supports debugging, testing, verification, resumability, and extensibility because every stage writes inspectable JSON.
 2. **Separate geometry from rendering.** Once the renderer was forbidden from computing positions, an entire class of bugs ("looks wrong, no idea which layer") disappeared. Every visual defect now has a defined home.
 3. **Real estates beat synthetic fixtures.** Every hard bug — hub-name attachment ordering, transit demotion, firewall-less hubs, self-loop private endpoints — came from real infrastructure quirks that a hand-written fixture would never have contained.
 4. **Determinism turned diagrams into audit evidence.** Reproducible, diffable outputs turned the diagrams from illustrations into audit evidence. Regulated-environment reviewers care more about "same input, same output" than about visual polish.
 5. **Geometric verification replaced visual inspection.** Layout regressions used to survive because a human glanced at a PNG and approved it. Since the verifier became a mandatory gate, no edge-through-container defect has shipped.
 6. **Naming conventions are load-bearing data.** Environment and DR classification derive from resource naming patterns. Where conventions were inconsistent, classification needed explicit pattern extensions. Documentation tooling exposes naming discipline — or its absence — within seconds.
 7. **Stdlib-only cores age well.** Three of four stages have zero third-party dependencies. Upgrades are non-events, security review is trivial, and the pipeline runs on locked-down machines where package installation is a negotiation.
 8. **Two-pass algorithms are worth the second loop.** Classification-then-attachment removed a whole category of ordering bugs. When in doubt, finish deciding before starting to link.
-

14. Roadmap — Unified Documentation and Deployment Toolchain

Status: vision, not yet implemented. Documented as the intended future state.

The documentation pipeline (this system) and a companion declarative deployment framework are two halves of one operational lifecycle: the deployment framework creates Azure infrastructure from inventory files; doc-generator documents existing infrastructure from snapshots. The companion document is *Universal Deployment Framework (UDF2) - Architecture Reference*. The intended convergence is a closed loop: deploy from inventory, document from snapshot, validate round-trip consistency between the two.

Area	Today (two tools)	Future (unified)
Inventory format	Deployment inventory JSON vs <code>snapshot.json</code>	Single canonical format, bidirectional conversion
Topology model	Independent classification logic	Shared topology library
Validation	Deploy preflight checks vs documentation compliance rules	One validation pass covering both
Architecture profiles	The Centralized Inspection profile and the Distributed Native profile supported independently	Single profile definition consumed by both

The binding constraint: each tool must remain fully usable standalone. The unified layer is an orchestration skin, not a rewrite.

Appendix A — CLI Reference

```
# Phase 1: collect inventory (assumes az login done)
make snapshot SNAPSHOT_ARGS="--subscription SUB_ID"
make snapshot-dry-run # validate prerequisites only

# Phase 2: Excel workbook from the latest snapshot
make docs

# Phase 3: diagrams from the latest snapshot (next-gen pipeline)
make diagrams
python3 run_pipeline.py snapshots/contoso-demo/snapshot.json --dpi 150
python3 run_pipeline.py snapshots/contoso-demo/snapshot.json --step 3 # resume from layout

# Verification
python3 scripts/verify_edge_routing.py output/contoso-demo/02-layout.json
python3 -m pytest tests/ -q # full suite: 382 tests

# Legacy drawio generator
python3 scripts/build_diagrams.py snapshots/contoso-demo/snapshot.json
```

Key CLI options of `run_pipeline.py`:

Option	Default	Purpose
snapshot	required	Path to snapshot.json
--name	derived from directory	Diagram name override
--output-dir	output/	Output base directory
--step	1	Resume from stage N (1-4)
--dpi	150	PNG rasterization DPI

End of document. All names are placeholders; all figures verified against the repository and re-measured on 2026-08-13.